

LLM Agents for Cross-Platform Prediction Market Arbitrage

Abhi Wadhwa
University of Southern California
abhiw@usc.edu

Summer 2026

Abstract

Prediction markets on Kalshi and Polymarket price the same real-world events under different naming conventions, fee structures, and settlement rules. Cross-platform arbitrage requires solving three chained problems: matching semantically equivalent contracts across platforms, detecting fee-adjusted pricing discrepancies from order book data, and executing trades under non-atomic settlement risk. I develop a formal treatment of cross-platform prediction market microstructure—fee models, arbitrage definitions, and the contract matching problem—and build a hybrid NLP pipeline (regex extraction, Sentence-BERT embeddings, rapidfuzz string matching, optional LLM verification) for cross-platform contract resolution. I evaluate the matching pipeline on a synthetic benchmark of 5,000 market pairs and report results. Real-data evaluation on live Kalshi and Polymarket markets, LLM agent experiments with actual API calls, and paper trading simulations are in progress and will be reported in a subsequent version.

1 Introduction

I got into prediction markets because of a football game. Chiefs–Bills, October 12, 2025. I was on my laptop at 2 AM—couldn’t sleep, bad habit—and I noticed Kalshi had Kansas City at 47 cents. Polymarket had them at 52. Five cents of spread on a contract that pays a dollar. Even after fees that’s about three cents per contract, free money in theory. So I bought YES on Kalshi, NO on Polymarket, made \$2.80 on a hundred contracts, and then sat there for a while thinking about how often this kind of thing happens.

Turns out: a lot. Prediction markets aren’t the academic curiosity they were ten years ago. Kalshi’s the first CFTC-regulated designated contract market for event contracts, and they now list thousands of markets on sports, politics, weather, economic data. Polymarket runs on Polygon and did \$22.88 billion in volume in 2025. Billion, with a B. That much money flowing through two different platforms with two different trader populations, two different fee structures, two different settlement mechanisms—of course there’s disagreement. And disagreement is just a polite word for arbitrage.

[Saguillo et al. \(2025\)](#) put a number on it: roughly \$40 million in realized cross-platform arbitrage profits between April 2024 and April 2025. Most of that was concentrated around high-attention events—presidential debates, NFL playoffs, that sort of thing. And that’s just the money somebody actually captured. The total addressable spread, counting opportunities that got missed because of latency or capital constraints or just nobody looking, has to be bigger.

But here’s the thing. Cross-platform prediction market arbitrage isn’t like equity arbitrage. There are three separate skills involved, and they share almost nothing.

NLP matching. Kalshi encodes events as structured tickers: `KXNFLGAME-25OCT12-KCBUF`. Polymarket uses free text: “Will the Kansas City Chiefs beat the Buffalo Bills on October 12, 2025?” You have to know that KC means Kansas City, that 25OCT12 means October 12, 2025, that NFLGAME implies a binary outcome. For politics and econ it gets worse. Kalshi might say “GDP Q3 2025 above 2.5%” and Polymarket phrases it “Will US GDP growth exceed 2.5% in 2025 Q3?” Same event. Totally different surface form.

Fee-adjusted pricing. Both platforms charge taker fees. Different formulas. Kalshi has this nonlinear thing with a ceiling function. Polymarket uses a continuous multiplier. A spread that looks good before fees can vanish after them. Get the arithmetic wrong and you lose money on what you thought was a sure thing.

Execution under uncertainty. This is the part that actually scared me. Prediction market arbitrage is non-atomic. You fill one leg on Kalshi. Then you go fill the other leg on Polymarket. In between—could be seconds, could be minutes—the price on the second platform moves. That gap is where the money disappears.

So: matching is a language problem. Fee calculation is a math problem. Execution is decision-making under sequential uncertainty. I like this decomposition because each subtask is independently measurable, the outcome is in dollars, and there’s a natural rule-based baseline for comparison. It’s a clean testbed for LLM agents.

The question I’m after isn’t whether LLMs can “do arbitrage.” That’s too vague. I want to know where in the pipeline they add value and where they don’t. My bet is that LLMs will be good at matching—it’s fundamentally about language understanding—and bad at fee calculation, because it’s fundamentally about arithmetic. I’m still testing this.

I make four contributions:

1. A formal treatment of cross-platform prediction market microstructure: fee models for both Kalshi and Polymarket, three types of arbitrage (cross-platform, implied same-platform, monotonicity violations), and the contract matching problem as entity resolution.
2. A hybrid NLP pipeline for cross-platform contract matching, evaluated on a synthetic benchmark with precision/recall analysis across market categories.
3. A synthetic benchmark of 5,000 cross-platform market pairs for development and unit testing of matching and arbitrage detection systems.
4. A framework for LLM agent evaluation—task decomposition, prompt design, rule-based baselines, and evaluation metrics—designed but not yet tested with real LLM API calls on live market data.

Here’s how the rest of the paper goes. Section 2 covers market microstructure, fees, and a formal treatment of arbitrage types. Section 3 is the system architecture—the hybrid matching pipeline and the order book model. Section 4 describes the LLM agent design and the rule-based baselines I’m comparing against. Section 5 is the synthetic benchmark and matching results on it. Section 6 is an honest status report: what’s verified, what’s preliminary, what I haven’t done yet. Section 7 is discussion.

2 Background and Market Microstructure

2.1 Binary Contracts

Definition 2.1 (Binary Contract). A binary contract on event E is a financial instrument that pays \$1 if E occurs and \$0 otherwise. A YES position at price $p \in (0, 1)$ profits $1 - p$ if E occurs

and loses p if E doesn't. A NO position at price q profits $1 - q$ if E doesn't occur and loses q if it does.

Both platforms trade binary contracts. That's where the similarity ends.

Kalshi is CFTC-regulated. A designated contract market. Tickers follow a structured convention: `KXNFLGAME-25OCT12-KCBUF` gives you the category (NFL game), date (October 12, 2025), and teams (Kansas City vs. Buffalo). They've got REST and WebSocket APIs authenticated via RSA-PSS signatures. Order books only report YES-side bids and asks—NO-side prices you have to infer.

Polymarket settles on Polygon. Events get condition IDs through the UMA oracle system. Three API layers: Gamma for market metadata, CLOB for order book data, Data for historical trades. Authentication is EIP-712 typed signatures. Their order books report YES and NO tokens as independent tradeable assets, which is a pretty different setup.

Here's a structural difference that matters more than it sounds like it should: Kalshi only gives you bid data for the YES side. If YES has a best bid of p cents, the implied NO ask is $100 - p$ cents. Polymarket reports YES and NO as separate tokens with their own order books. Nothing forces those prices to sum to exactly \$1 at any given moment. Which, yes, creates same-platform arbitrage opportunities all by itself.

2.2 Fee Models

Fees eat arbitrage. A fraction-of-a-cent error in your fee model can turn a profitable trade into a losing one. The two platforms don't even use similar formulas.

Definition 2.2 (Kalshi Taker Fee). For a trade of C contracts at price P (in dollars, $P \in (0, 1)$), the Kalshi taker fee in cents is:

$$f_K(C, P) = \lceil 0.07 \cdot C \cdot P(1 - P) \rceil \text{ cents.} \quad (1)$$

The ceiling function $\lceil \cdot \rceil$ means every trade pays at least 1 cent in fees, regardless of size.

Definition 2.3 (Polymarket Taker Fee). For a trade of Q tokens at price P , the Polymarket taker fee in USDC is:

$$f_P(Q, P) = \gamma \cdot Q \cdot P(1 - P), \quad (2)$$

where γ is a category-dependent fee rate, approximately 0.02 for sports markets.

They both have the $P(1 - P)$ kernel. But Kalshi's ceiling function introduces a discontinuity that screws you on small trades.

Proposition 2.4. *Both fee functions are maximized at $P = 0.5$ and vanish as $P \rightarrow 0$ or $P \rightarrow 1$.*

Proof. The function $g(P) = P(1 - P)$ has derivative $g'(P) = 1 - 2P$, which equals zero at $P = 1/2$. Since $g''(P) = -2 < 0$, the point $P = 1/2$ is a maximum. At the boundaries, $g(0) = g(1) = 0$. Both f_K and f_P are non-negative scalar multiples of $g(P)$ (up to the ceiling in f_K), so they inherit this behavior. $\square$

Remark 2.5. So the fees are fattest right where arbitrage lives. Mid-probability events—the ones near $P = 0.5$ —attract the most disagreement between platforms. Diverse opinions, heavy trading, big price discrepancies. But those exact same events charge you the most in fees. It's a moat. The market structure itself protects market makers from people like me.

Some concrete numbers. Buy 100 contracts at $P = 0.50$ on Kalshi: $\lceil 0.07 \times 100 \times 0.25 \rceil = \lceil 1.75 \rceil = 2$ cents. Polymarket with $\gamma = 0.02$: $0.02 \times 100 \times 0.25 = 0.50$ USDC. Kalshi’s fee is four times bigger at the midpoint. At $P = 0.10$, Kalshi charges $\lceil 0.07 \times 100 \times 0.09 \rceil = \lceil 0.63 \rceil = 1$ cent, Polymarket charges $0.02 \times 100 \times 0.09 = 0.18$ USDC. The ratio narrows at extreme prices, but Kalshi is always at least as expensive because of the ceiling.

2.3 Arbitrage Types

Three types of arbitrage. Different structure, different risk, different detection difficulty.

Definition 2.6 (Direct Cross-Platform Arbitrage). For the same event E , suppose Platform A offers YES at ask price a and Platform B offers NO at ask price b . The gross edge per contract is:

$$e_{\text{gross}} = 1 - a - b. \quad (3)$$

The net edge after fees for n contracts is:

$$e_{\text{net}}(n) = 1 - a - b - \frac{f_A(n, a)}{n} - \frac{f_B(n, b)}{n}. \quad (4)$$

An arbitrage opportunity exists when $e_{\text{net}}(n) > 0$ for some $n \geq 1$.

Definition 2.7 (Implied Same-Platform Arbitrage). On a single platform, if the best YES ask p_{yes} and best NO ask p_{no} satisfy

$$p_{\text{yes}} + p_{\text{no}} < 1 - \frac{f(n, p_{\text{yes}}) + f(n, p_{\text{no}})}{n} \quad (5)$$

for some n , then buying both YES and NO guarantees a profit.

Same-platform arb is rare on Kalshi—their centralized exchange mechanically enforces $p_{\text{yes}} + p_{\text{no}} \geq 1$. On Polymarket it’s common. YES and NO tokens trade independently, so the two sides can decouple during volatile news events. I’ve seen it happen during earnings releases where the spread blows out for ten or fifteen seconds.

Definition 2.8 (Monotonicity Violation). For a set of threshold markets on the same underlying variable X with thresholds $T_1 < T_2 < \dots < T_k$, the prices must satisfy:

$$\mathbb{P}(X > T_1) \geq \mathbb{P}(X > T_2) \geq \dots \geq \mathbb{P}(X > T_k). \quad (6)$$

A violation $\mathbb{P}(X > T_i) < \mathbb{P}(X > T_{i+1})$ for some i signals mispricing.

You see these in economic threshold markets. Kalshi lists five markets on Q3 GDP growth: above 1%, above 1.5%, above 2%, above 2.5%, above 3%. If “above 2.5%” is trading at 45 cents while “above 2%” sits at 42 cents, that’s wrong. The less restrictive threshold has to be priced weakly higher. These violations tend to be small—1 to 3 cents—and they don’t last.

Remark 2.9 (Non-Atomic Settlement Risk). This isn’t futures arbitrage where both legs fill at once on one exchange. In prediction markets the legs execute sequentially. I buy YES on Kalshi, then I have to go buy NO on Polymarket. Between those fills—call it δ seconds—the Polymarket price can move. If the NO price jumps by more than the original edge during that gap, my “arbitrage” is now a loss. This is why real profits are smaller than theoretical edge. You’re always racing the clock, and the clock doesn’t care about your spreadsheet.

2.4 The Matching Problem

None of the arbitrage detection matters if I can't figure out which Kalshi market corresponds to which Polymarket market. This is entity resolution, plain and simple.

Formally: let $\mathcal{K} = \{k_1, k_2, \dots, k_m\}$ be the set of active Kalshi tickers and $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ the set of active Polymarket events. I want all pairs $(k_i, p_j) \in \mathcal{K} \times \mathcal{P}$ that refer to the same real-world event.

It's not one-to-one. A single Polymarket event ("Will the Chiefs beat the Bills?") might correspond to multiple Kalshi tickers if Kalshi lists both game outcome and point spread. Going the other way, a Kalshi ticker for "Will inflation exceed 3% in 2025?" might have no Polymarket equivalent at all.

What makes the matching hard:

- **Naming conventions.** Kalshi uses structured abbreviations (KC for Kansas City, BUF for Buffalo). Polymarket spells everything out, sometimes with articles ("the Kansas City Chiefs").
- **Temporal encoding.** Kalshi does 25OCT12 (YYMMDD). Polymarket does whatever it feels like: "October 12, 2025" or "Oct 12" or "10/12/25."
- **Granularity.** Kalshi might list 10 threshold levels for GDP. Polymarket lists 5 different ones. You have to find the overlap.
- **Ambiguity.** Political events are the worst. "Will Biden comment on the debate?" versus "Will the President respond to the presidential debate controversy?"—same event? Maybe. Depends on context and timing. I genuinely don't know sometimes, and I'm a human.

Example 2.10. The ticker KXNFLGAME-25OCT12-KCBUF decomposes as:

- KX: Kalshi exchange prefix.
- NFLGAME: category (NFL game outcome).
- 25OCT12: date (October 12, 2025).
- KC: Kansas City Chiefs.
- BUF: Buffalo Bills.

The corresponding Polymarket event is: "Will the Kansas City Chiefs beat the Buffalo Bills on October 12, 2025?" Matching these requires a lookup table from team abbreviations to full names, date parsing, and the understanding that "NFLGAME" implies a binary win/loss outcome.

The matching problem is the gate. Miss a match and you miss money. False-match and you trade two unrelated events against each other. That's not arbitrage. That's just gambling with extra steps.

3 System Architecture

3.1 Hybrid Matching Pipeline

The matching system runs candidates through four stages. Easy matches die early. Hard ones propagate to more expensive layers. I like this design because my wallet likes this design.

Stage 1: Rule-Based Extraction. Regex patterns rip Kalshi tickers apart into structured fields: category, teams/entities, date, threshold if applicable. Polymarket descriptions get keyword extraction—named entity recognition pulls out teams, dates, numerical thresholds. Both sides produce a canonical tuple (c, e_1, e_2, d, t) where c is category, e_1, e_2 are entities, d is date, t is threshold. Exact field matches produce high-confidence pairs right away. On the synthetic benchmark, about 60% of true matches get caught here. Mostly sports, where both sides name things in a structured way.

Stage 2: Sentence-BERT Embedding. Whatever’s left unmatched gets normalized—Kalshi tickers have their abbreviations expanded from a lookup table, Polymarket descriptions get lowercased with articles removed—then both go through all-MiniLM-L6-v2 (Reimers and Gurevych, 2019). Cosine similarity above 0.85 means it’s a match. This catches the cases where field extraction failed: weird abbreviations, non-standard dates, events that just don’t decompose cleanly into the (c, e_1, e_2, d, t) schema.

Stage 3: Fuzzy String Fallback. Residual candidates—similarity between 0.60 and 0.85—get processed with rapidfuzz (Bachmann, 2021) token-set-ratio. Handles typos, minor rewording, and cases where the embedding model gave moderate similarity to pairs that genuinely match. Threshold is 80% token-set-ratio.

Stage 4: LLM Verification. The ambiguous zone: candidates with token-set-ratio between 80% and 95%, or SBERT similarity between 0.70 and 0.90. These get shipped to an LLM. The prompt includes both descriptions and asks for a binary match/no-match judgment plus a one-sentence explanation. This is the expensive stage—roughly $50\times$ the latency of Stage 2—so it only sees the tail. On the synthetic benchmark about 8% of candidate pairs reach it.

Each stage rejects obvious non-matches before the next one fires. The LLM only gets the stuff that the cheaper methods couldn’t figure out.

3.2 Order Book Model

I model each side of each market as an L2 order book: sorted arrays of (price, volume) tuples for bids and asks.

$$\text{Book}_{\text{ask}} = \{(p_1, v_1), (p_2, v_2), \dots, (p_L, v_L)\} \quad \text{with } p_1 < p_2 < \dots < p_L, \quad (7)$$

where L is the number of price levels. Each level is a standing limit order. To fill n contracts you walk up the book: eat v_1 contracts at p_1 , then v_2 at p_2 , keep going until you’re filled or the book’s empty.

Effective fill price for n contracts is the volume-weighted average:

$$\bar{p}(n) = \frac{\sum_{i=1}^k p_i \cdot \min(v_i, r_i)}{\sum_{i=1}^k \min(v_i, r_i)}, \quad (8)$$

where $r_1 = n$ and $r_{i+1} = r_i - \min(v_i, r_i)$, with k being the first level where $r_k \leq v_k$.

Cross-platform synchronization is a headache I haven’t fully solved. Kalshi’s WebSocket updates show up every 100–500ms. Polymarket’s CLOB can lag 1–3 seconds during heavy volume. Any book older than 30 seconds is garbage for arbitrage purposes—prices move 2–3 cents in that window.

3.3 Fee-Adjusted Edge Calculation

The net edge for a cross-platform trade of n contracts, buying YES on Platform A at effective price $\bar{a}(n)$ and NO on Platform B at effective price $\bar{b}(n)$:

$$e_{\text{net}}(n) = 1 - \bar{a}(n) - \bar{b}(n) - \frac{f_A(n, \bar{a}(n))}{n} - \frac{f_B(n, \bar{b}(n))}{n}. \quad (9)$$

This is concave in n , and for two separate reasons. First, $\bar{a}(n)$ and $\bar{b}(n)$ are weakly increasing in n —walking up the book means worse prices. Second, Kalshi’s ceiling function makes the per-contract fee $f_K(n, P)/n$ weakly decreasing in n , because the 1-cent overhead from the ceiling gets spread over more contracts. There’s a tension: bigger size improves fee efficiency but worsens fill prices.

Optimal position size n^* satisfies:

$$n^* = \arg \max_{n \in \{1, \dots, N_{\text{max}}\}} n \cdot e_{\text{net}}(n), \quad (10)$$

where N_{max} is the minimum depth available across both books. I don’t actually optimize over every possible size. I compute $e_{\text{net}}(n)$ for $n \in \{1, 10, 25, 50, 100\}$ and pick the winner. The surface is smooth enough that this coarse grid works fine. I spent an embarrassing amount of time writing a continuous optimizer before realizing five grid points gave the same answer every time.

Minimum edge threshold: 2%. If $e_{\text{net}}(n^*) < 0.02$, skip it. That 2% covers execution risk (the non-atomic gap from Remark 2.9), slippage from stale books, and the chance that the matcher gave me a false positive.

4 LLM Agent Design

4.1 Task Decomposition

I split the arbitrage pipeline into three tasks. Each one is testable on its own.

Definition 4.1 (Task T_1 : Match Verification). Given two market descriptions (d_K, d_P) —one from Kalshi, one from Polymarket—output $\hat{y} \in \{0, 1\}$ indicating whether they refer to the same real-world event.

Definition 4.2 (Task T_2 : Trade Decision). Given an arbitrage opportunity $o = (\text{match pair, order books, edge, fees})$ and current portfolio state $s = (\text{positions, capital, daily PnL})$, output a decision $\hat{d} \in \{\text{trade, skip}\}$ and, if trading, a position size $\hat{n}$.

Definition 4.3 (Task T_3 : Risk Assessment). Given portfolio state s and a set of pending opportunities $\{o_1, \dots, o_k\}$, output risk metrics: estimated portfolio variance, maximum loss scenario, and a recommendation to continue trading or pause.

T_1 is language. T_2 is quantitative decision-making. T_3 is a monitoring task that gates the other two. In a fully autonomous agent all three run continuously; my evaluation framework tests them separately first, then together.

4.2 Agent Architecture

For each task the LLM gets a structured prompt with everything it needs. I’m not reproducing the full prompts here—they’re in the code repo—but I’ll describe the design choices that matter.

T_1 : the prompt shows the Kalshi ticker expanded into natural language (using the abbreviation lookup table) next to the Polymarket description. “Do these two descriptions refer to the same event? Answer YES or NO. Then explain your reasoning in one sentence.” I log the explanation but don’t use it algorithmically.

T_2 : the prompt includes matched pair descriptions, both order books (top 5 levels), computed gross and net edge, fee breakdown, current positions, available capital, the day’s running PnL. The LLM outputs a JSON object: `{‘action’: ‘trade’/‘skip’, ‘size’: int, ‘reason’: string}`. The reason field is for interpretability, mostly for me when I’m debugging at midnight.

Here’s the test I’ve embedded in T_2 : can the LLM correctly verify the fee computation? The prompt gives raw inputs and asks the agent to confirm or reject the precomputed fee:

Kalshi fee for 100 contracts at $P = 0.47$: computed as $\lceil 0.07 \times 100 \times 0.47 \times 0.53 \rceil = \lceil 1.7402 \rceil = 2$ cents. Is this correct?

Yes, it’s correct. I expect this to be where LLMs fall apart. Multi-step floating-point multiplication into a discrete ceiling function is exactly the kind of computation where approximate reasoning breaks down. But I don’t have real error rates yet. The experiments in Section 6.3 will produce those.

T_3 : the prompt presents everything—all open positions, entry prices, current market prices, total exposure. The LLM estimates worst-case loss (every position goes to zero on the losing side) and recommends whether to keep trading or stop for the day. Probably the easiest of the three tasks for an LLM. It’s mostly qualitative reasoning about concentration.

4.3 Rule-Based Baseline

No LLM at all. Pure code.

Matching. Stages 1–3 of the hybrid pipeline (Section 3.1) with tuned thresholds: SBERT similarity ≥ 0.85 , rapidfuzz token-set-ratio $\geq 80\%$. Stage 4 doesn’t run.

Trading. If $e_{\text{net}}(n^*) > 0.02$ and there’s enough depth (≥ 50 contracts at best price on both sides), execute. Position sizing is fractional Kelly:

$$n = \min \left(\left\lfloor 0.25 \cdot \frac{e_{\text{net}}}{\sigma^2} \right\rfloor, N_{\text{max}} \right), \quad (11)$$

where σ^2 is the estimated variance of the outcome (for a binary at price p , $\sigma^2 = p(1-p)$), and N_{max} is the position limit. The 0.25 multiplier is quarter-Kelly—standard practice for trading variance against expected growth. Position limits: \$100 per market, \$10,000 total exposure.

Risk. Hard stop: if daily PnL drops below -5% of capital, stop trading. Max 30% of capital in any single category. These rules are dumb and fast and that’s the point. No NLP. Microsecond execution.

4.4 Evaluation Metrics

Matching: precision (P), recall (R), F1 ($F1 = 2PR/(P + R)$). Globally and per category.

Trading: the usual portfolio stuff.

- Cumulative PnL: total dollars earned net of all fees.
- Sharpe ratio: $S = \bar{r}/\sigma_r$, where $\bar{r}$ is mean daily return and σ_r is daily return standard deviation. Annualized by multiplying by $\sqrt{252}$.

- Maximum drawdown: largest peak-to-trough decline in cumulative PnL.
- Trade count and error count.

Decision quality is the fraction of decisions that agree with the ex-post optimum:

$$\text{DQ} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{sign}(\hat{d}_i) = \text{sign}(d_i^*)], \quad (12)$$

where $\hat{d}_i$ is the agent’s decision and d_i^* is the one that would’ve been optimal given realized outcomes. Noisy metric—some right decisions are just unlucky—so I report it alongside PnL, which integrates both decision quality and sizing accuracy.

Status note. The evaluation framework is fully implemented. Prompts are written, the rule-based baseline runs, metrics compute correctly on synthetic data. What’s missing: real LLM API calls (GPT-4, Claude, etc.) on live market data. Section 6.3 has the plan.

5 Synthetic Benchmark

I built a synthetic benchmark for development and unit testing. This section describes how the data was generated and reports matching results on it. These results tell me whether the pipeline’s internals work—whether the stages are correct and compose properly. They don’t tell me how the pipeline handles real Kalshi and Polymarket markets, where naming is messier, categories more diverse, and the difficulty distribution different. Real-data evaluation is in Section 6.2 and Section 6.3.

5.1 Synthetic Data Generation

Two reasons to start with synthetic data instead of live markets. Reproducibility: live markets change second by second, so experiments aren’t repeatable. Control: I can independently vary matching difficulty, arbitrage edge size, and order book liquidity.

5,000 market pairs. Category breakdown: 50% sports (2,500 pairs), 20% crypto (1,000), 15% politics (750), 15% economics (750). This roughly tracks the volume distribution on both platforms.

For each true match I generate:

- A Kalshi-style ticker following the structured convention (`KXCATEGORY-YYMMDD-ENTITIES`).
- A Polymarket-style description in natural language.
- Two order books, 10 price levels each. Spreads drawn uniformly from 1–5 cents. Depth at each level from a log-normal distribution with mean 50 contracts.

Negative pairs come from corrupting one field of a true match: swap teams (“Chiefs vs Bills” → “Chiefs vs Ravens”), shift the date by a day, change the threshold (“above 2.5%” → “above 3.0%”), or replace the category entirely. These are hard negatives—most fields match, but one critical field differs. About 40% of the 5,000 pairs are true matches; 60% are negatives, half hard, half easy (completely different events).

Table 1: Cross-platform matching results on the **synthetic** benchmark (5,000 market pairs). Stage 4 uses a mock LLM verifier, not real LLM API calls. These numbers test pipeline logic, not real-world matching performance.

Method	Precision	Recall	F1
Regex only	0.891	0.723	0.798
Regex + SBERT	0.931	0.912	0.921
Regex + SBERT + Fuzzy	0.931	0.947	0.939
Full pipeline (+mock LLM)	0.978	0.943	0.960
Mock LLM only	0.854	0.912	0.882

5.2 Matching Results on Synthetic Data

Five matching configurations, tested on the synthetic benchmark:

1. **Regex only**: Stage 1 alone.
2. **Regex + SBERT**: Stages 1–2.
3. **Regex + SBERT + Fuzzy**: Stages 1–3 (full pipeline, no LLM).
4. **Full pipeline**: All four stages (mock LLM verifier applying string-similarity heuristics to simulate Stage 4 decisions).
5. **Mock LLM only**: Skip all rule-based stages; the mock verifier processes every pair.

Big caveat: Stage 4 here is a mock LLM, not real GPT-4 or Claude. It’s a string-similarity heuristic pretending to be an LLM. The numbers below test pipeline architecture and Stages 1–3. They say nothing about what a real LLM would do on Stage 4. Real LLM evaluation is in Section 6.3.

The first three rows are what actually matters here. The hybrid pipeline without any LLM—Regex + SBERT + Fuzzy—hits 93.9% F1 on synthetic data. Precision and recall are balanced. Each stage earns its keep: regex alone gets 79.8% F1 by catching the structured easy stuff. SBERT jumps it to 92.1%, a 12.3-point gain, by handling cases where field extraction choked. Fuzzy matching adds another 1.8 points on the remaining near-misses.

The last two rows: the mock verifier. As Stage 4 of the cascade it boosts precision from 93.1% to 97.8%. Used alone it’s mediocre—88.2% F1—because it lacks the structured extraction the other stages provide. Good for validating the cascade architecture. Don’t read anything into it about real LLM performance.

Category-level performance on synthetic data: sports is easiest (structured abbreviations map cleanly), politics is hardest (paraphrase, ambiguity), crypto and econ fall in between. But I’m pretty sure synthetic data understates the difficulty of real political events, where descriptions are wilder and more context-dependent.

5.3 Ablation: Pipeline Stage Contribution

I measured each stage’s contribution on the synthetic data along three axes: F1 improvement, latency cost, and how many candidate pairs it processes (the cascade filter effect).

The cascade filters hard. Stage 1 resolves about 60% of true matches. Stage 2 catches another 30%. Stage 3 handles about 8%. Only the remaining 2% would reach Stage 4 in the full pipeline. Latency is correspondingly skewed: regex runs in microseconds, SBERT encoding is 2–5ms per pair,

fuzzy matching is comparable, and LLM verification with a real model would average around 800ms per pair. The cascade is what makes the expensive stage practical. You can’t afford 800ms per pair on everything. You can afford it on 2%.

6 Results and Status

I’m going to be blunt about what’s done and what isn’t. No hedging. If you skim one section of this paper, skim this one.

6.1 Verified Components

These components are implemented, tested, and correct:

Fee models. The Kalshi formula (Definition 2.2) and Polymarket formula (Definition 2.3) are implemented and verified against platform API docs and actual fee schedules. The ceiling-function behavior produces correct outputs for all tested inputs, including edge cases at extreme prices (P near 0 or 1) and tiny trade sizes ($C = 1$) where the ceiling dominates.

Arbitrage definitions. All three types (cross-platform, implied same-platform, monotonicity violations) are correctly formalized and implemented as detection functions. Given correct prices, they find the right opportunities. Net edge calculation (Eq. 9) handles both fee models and book depth.

Matching pipeline (Stages 1–3). Rule-based extraction, SBERT embedding, fuzzy string matching—all run correctly. 93.9% F1 on the synthetic benchmark without LLM (Table 1). All seven Python files run without errors, which I’m probably more proud of than I should be.

Synthetic benchmark. 5,000 pairs, correct category distribution, correct difficulty profile. Does what it’s supposed to: development, debugging, unit testing.

Mathematical framework. Fee model analysis (Proposition 2.4), the fee moat observation (Remark 2.5), order book model, position sizing—these are correct descriptions of real systems. They’re derived from platform specs. No empirical validation needed.

6.2 Preliminary Results: Live Market Data

I built a data fetcher that hits the Kalshi API (`api.elections.kalshi.com/trade-api/v2`) and the Polymarket Gamma API (`gamma-api.polymarket.com`), normalizes the responses, and runs the three-stage matching pipeline. Both endpoints are public, no auth required for read access. I ran this on May 25, 2026. Here’s what came back.

Market inventory. Kalshi: 813 active markets. Polymarket: 46 active markets (filtered to sports and crypto with active order books). The asymmetry is itself a finding. Polymarket lists far fewer markets at any given time.

Kalshi’s structural shift. This is the thing I didn’t expect. Of 813 Kalshi markets, 782—that’s 96.2%—were multi-leg parlay contracts under the `KXMVESPORTSMULTIGAMEEXTENDED` ticker family. These are compound bets: “OKC wins by over 3.5 points AND over 219.5 points scored.” Only 31 markets (3.8%) were simple binary outcomes. My entire system architecture assumes simple binary matching—one Kalshi event maps to one Polymarket event—but Kalshi’s current inventory is almost entirely parlays with no single-event Polymarket equivalent. I stared at my terminal for a while after discovering this.

Matching results. 14 candidate matches, all sports. The highest confidence was 0.564: an OKC Thunder parlay matched to “Will the Oklahoma City Thunder win the NBA Western Conference Finals?” All 14 were structurally wrong—multi-leg Kalshi parlays paired with single-outcome

Polymarket markets. The pipeline did assign low confidence scores (none above 0.57), but didn't reject them outright because the team name overlap creates real semantic similarity. "Oklahoma City" is "Oklahoma City" whether it's in a parlay or not.

Order book analysis. Most of the 14 candidates had empty or illiquid Kalshi books—best bid = 0, best ask = 1.00. One showed a Kalshi ask of \$0.062 against a Polymarket bid of \$0.62. Nominal gross edge: 55.8 cents. Incredible, right? No. The Kalshi contract is a multi-leg parlay (OKC wins by 2.5+ AND total over 219.5 AND total over 230.5). The Polymarket contract is a simple series-winner bet. They're not the same event. The price difference reflects the lower probability of the compound outcome, not any kind of market inefficiency. I almost got excited. I shouldn't have.

Key finding. Live data reveals a structural mismatch between current platform inventories that my synthetic benchmark completely missed. Cross-platform arbitrage needs matching pairs of simple binary contracts. Kalshi's shift to multi-leg parlays means most of its volume is in contracts with no Polymarket equivalent. The opportunity set might be smaller than [Saguillo et al. \(2025\)](#)'s \$40 million figure suggests—that covered a period when both platforms listed more single-outcome events.

This doesn't kill the pipeline or the LLM evaluation framework. But it changes what deployment looks like. A production system would need a parlay decomposition stage: parse multi-leg Kalshi contracts into individual components, match each one separately. Haven't built that yet.

6.3 Planned Experiments

Five experiments left, roughly priority-ordered:

1. **Hand-labeled matching benchmark on live markets ($N \geq 200$ pairs).** Fetch active markets from both platforms, hand-label matches and non-matches across categories, evaluate all five pipeline configurations on real data. This is the single most important missing piece. Synthetic matching performance doesn't transfer reliably to real naming patterns.
2. **LLM agent evaluation with real API calls.** Run GPT-4 and Claude on all three tasks (T_1 : match verification, T_2 : trade decision, T_3 : risk assessment) using real market data. Measure the actual fee calculation error rate. I think LLMs will botch the Kalshi ceiling function but I don't have a number.
3. **Paper trading simulation with live order book data.** Run both agents—rule-based and LLM—on a multi-week simulation using real order book snapshots. This requires solving cross-platform synchronization (matching timestamps between Kalshi WebSocket data and Polymarket CLOB snapshots). Not trivial.
4. **Fee calculation error rate measurement.** Present real fee problems (actual prices and quantities from live markets) to multiple LLMs. Measure arithmetic error rate, ceiling omission rate, unit confusion rate. This is a focused slice of item 2 and can run independently.
5. **Full backtest with historical data.** If I can get historical order book data (Kalshi has some through their data API; Polymarket requires scraping), run a retrospective simulation over 3–6 months. Estimate PnL, Sharpe, drawdown for both agent types.

Until these experiments run, what I can claim is limited: pipeline architecture is sound, fee math is correct, matching works on synthetic data. Whether the system finds real arbitrage, and whether LLMs help or hurt at each stage—I don't know yet.

7 Discussion

7.1 What the Synthetic Results Tell Us

The synthetic benchmark validates architecture. Each stage earns its place. Regex handles structured easy cases. SBERT catches semantic equivalences regex can't. Fuzzy matching picks up near-misses. The cascade filters aggressively enough that the LLM stage, once it's real, will only see a small fraction of pairs. Good. You don't want to pay for 800ms per pair when regex could've answered in microseconds.

93.9% F1 on synthetic data without an LLM. Sounds nice. I'm skeptical of it. Synthetic data is generated by rules, and the pipeline is rule-based. There's some circularity: I designed the hard negatives by corrupting single fields, which is exactly the kind of difficulty that field-matching is built to detect. Real hard negatives might be hard in ways I didn't think of—different categories that happen to share entity names, or two different games between the same teams on different dates. A regular season game vs. a playoff game. That kind of thing.

The real test is live data. I expect performance to drop, especially on politics and economics markets where Polymarket descriptions are more varied than my synthetic templates. Sports matching should hold up because the naming really is structured on both sides. NFL teams don't rebrand between game weeks.

7.2 Expected LLM Behavior

I have a hypothesis. No data. Let me state it so the experiments can kill it.

Matching (T_1): LLMs should add value. Especially on politics and economics where paraphrase and implicit references are common. “Will the former president attend the debate?” matching “Trump debate attendance”—that mapping requires world knowledge the rule-based pipeline doesn't have. The gain should be largest on the hard tail.

Fee calculation (T_2): LLMs should fail. The Kalshi formula is multi-step floating-point multiplication followed by a ceiling function. The literature on LLM arithmetic ([Dziri et al., 2023](#); [Qian et al., 2022](#)) says this is where errors compound. The ceiling adds a discrete cliff to a smooth computation. I predict 5–15% error rates on Kalshi fee computation, below 3% on Polymarket's simpler formula. But these are guesses. I want to be wrong about the 5–15%. I don't think I will be.

State tracking (T_3): LLMs should handle qualitative risk assessment okay—spotting concentration, flagging overexposure. But I expect them to lose track of precise capital across sequential decisions because each prompt call is stateless. The portfolio state is just text in a prompt. The rule-based agent tracks capital with a running counter. No contest.

My headline prediction: LLMs help at the NLP layer, hurt at the quantitative layer. Net contribution to PnL depends on the ratio of language to math in the pipeline. This pipeline has more math than language.

7.3 Design Implications

Even without real results the design points somewhere specific. LLMs for the language parts—matching, extracting information from news, interpreting ambiguous settlement rules. Deterministic code for everything with numbers—fee computation, edge calculation, position sizing, risk limits, order book parsing.

This lines up with the tool-use paradigm from [Schick et al. \(2023\)](#): LLMs reason and plan, tools compute. Here the “tools” are the fee calculator, the order book parser, the Kelly sizing formula.

The LLM’s job is deciding whether two events match and interpreting qualitative information—news, settlement rules, regulatory changes. Everything that fits in a formula should be a formula.

A concrete architecture:

1. **Ingestion layer:** rule-based. Parse API responses, build order books, compute derived quantities.
2. **Matching layer:** hybrid. Regex + SBERT + fuzzy for the easy 92% of pairs, LLM for the 8% ambiguous tail.
3. **Detection layer:** rule-based. Fee computation, edge calculation, opportunity scoring.
4. **Execution layer:** rule-based. Kelly sizing, position limits, stop-losses.
5. **Monitoring layer:** LLM. Summarize daily performance, flag weirdness, interpret news affecting open positions.

LLM at stages 2 and 5. Stages 1, 3, and 4 are pure code. More complex than “just use an LLM for everything,” but should be more reliable and cheaper. Should be. Whether it actually is—well, that’s what the experiments are for.

7.4 Limitations

The biggest limitation is obvious. Right now this is a design paper with synthetic validation. The theory works. The code runs. The pipeline handles fake data. But I don’t have the numbers that matter—real matching accuracy, real LLM error rates, real PnL.

My synthetic data understates difficulty in specific ways. I generate order books from parametric distributions: log-normal depth, uniform spreads. Real order books have fat tails, clustering, autocorrelation. Real markets are adversarial—market makers adapt to arb flow, widen spreads when they detect it, use information from one platform’s book to update the other. My fake data doesn’t capture any of that. Real markets are mean. My simulations are polite.

I’ve tested against one mock LLM and zero real ones. GPT-4, Claude, Gemini—they’ve all got different training distributions. Maybe one of them handles the ceiling function fine. I doubt it, but I don’t know. Multi-model evaluation is planned.

No execution risk modeling with real data. In production you’d hit rejected orders, partial fills, platform outages, API rate limits. Polymarket’s CLOB has minimum order sizes and tick sizes I haven’t tested against. I’m aware this matters. Haven’t gotten there yet.

And there’s the regulatory question looming over all of it. Kalshi is CFTC-regulated. Polymarket’s primary interface operates from outside the United States, regulatory status ambiguous. Cross-platform arb between a regulated and unregulated venue raises legal questions for anyone in the U.S. that I don’t address here. The technical pipeline works regardless. Deploying it is a different conversation, and not one I’m qualified to have.

Acknowledgments

Thanks to the prediction markets crowd on Twitter for putting fee structures, API docs, and arb strategies out in public. The open discussion of market microstructure on Kalshi and Polymarket is the reason this project exists.

References

- Max Bachmann. Rapidfuzz: Rapid fuzzy string matching. <https://github.com/maxbachmann/RapidFuzz>, 2021.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D Hwang, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Jing Qian, Hong Wang, Zekun Li, Shiyang Li, and Xifeng Yan. Limitations of language models in certain reasoning tasks. *arXiv preprint arXiv:2210.06726*, 2022.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.
- Pedro Saguillo, Seyed Hossein Ghafouri, Lucianna Kiffer, and Guillermo Suarez-Tangil. Arbitrage in prediction markets. *arXiv preprint arXiv:2508.03474*, 2025.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.